

AI RAG CHATBOT

System Architecture Spec & Technical API Manual

Complete System Documentation

Integrated architecture blueprint, pipeline maps, security analysis, and API manual.

CORE TECHNOLOGY STACK:

- LLM: LLaMA 3.3 70B Versatile (via Groq Cloud SDK)
- Vector Store: Local Persistent ChromaDB Indexing
- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- Relational Storage: SQLite Credentials & Message Logs
- Client Interface: Customized Streamlit Web UI

Published: July 02, 2026 | Version 1.0.0 Stable

1. Project Overview & Explanation

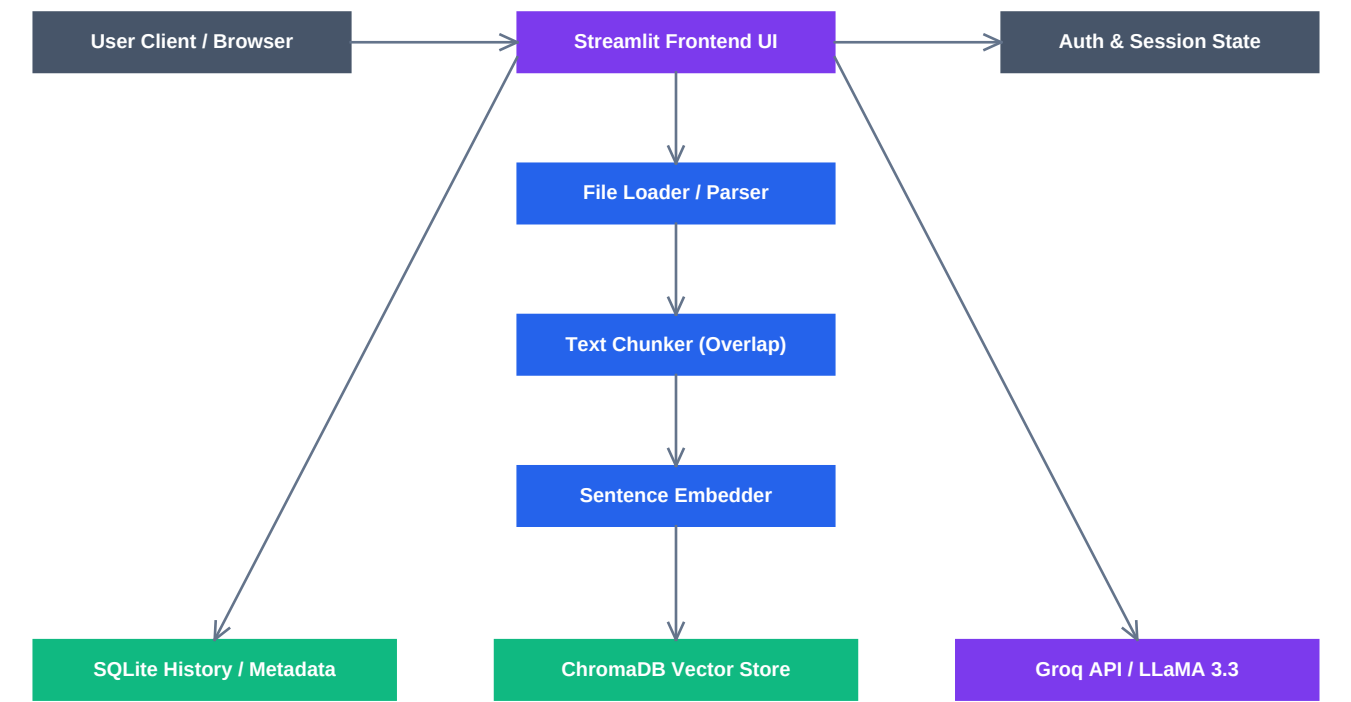
The AI RAG Chatbot is a secure, high-performance document intelligence assistant. It utilizes Retrieval-Augmented Generation (RAG) to allow users to upload multiple documents (including PDF, DOCX, TXT, MD, and CSV files) and query their contents interactively. Traditional LLMs are limited by their training cutoff dates and lack access to private, proprietary information. This system overcomes those limits by anchoring LLM responses in real-time retrieved facts, eliminating hallucinations and ensuring factual accuracy based solely on the uploaded files.

What makes this implementation exceptional is its core performance layer: it connects to the ultra-fast Groq Inference API to load LLaMA 3.3 70B versatile. Groq's custom hardware architecture processes LLM inferences at speeds up to 300+ tokens per second, creating a seamless, natural dialogue flow. The application also features a multi-user authentication model to ensure data sovereignty. Users have isolated workspaces: both their vector database records (ChromaDB) and chat history records (SQLite) are segregated and secure.

KEY ARCHITECTURAL HIGHLIGHTS:

- Zero-Latency Chunk Retrieval: ChromaDB persistent vector space optimized using cosine similarity.
- Multi-Format Parser: Unified parsing of tabular (CSV), structured (DOCX/PDF), and flat files (TXT/MD).
- Fail-Safe Context Manager: Trims and pairs chat histories automatically to prevent token-overflows.
- Cryptographic Auth Security: Bcrypt hashing guarantees secure credentials storage in SQLite.

2. System Flow Architecture



Component Operations Map:

Component	Technology	Primary Responsibility
Streamlit UI	Python Streamlit	Orchestrates chat inputs, layout cards, and multi-file uploads.
Parser / Loader	pdfplumber, docx	Reads and extracts texts from PDF, DOCX, CSV and TXT files.
Sentence Embedder	SentenceTransformers	Vectorizes word splits into 384-dimensional dense arrays.
ChromaDB Store	Local Vector DB	Performs cosine similarity lookups for relevant chunks.
SQLite Engine	SQLite3 Client	Houses credential hashes, document files registry, and message logs.
Groq Client	Groq SDK	Directs token streams, conversation pairs, and error fallback states.

Pipeline Ingestion & Security Model

1. Ingestion Pipeline (File Upload)

During document upload, file parser loaders (pdfplumber/python-docx/pandas) read raw binary structures. The string content is sent to the chunker to split it into 500-word segments (with 50-word margins). These text blocks are then converted to 384-d dense vectors by SentenceTransformer. The database helper logs files tracking UUIDs in SQLite, and ChromaDB indexes the vectors alongside reference sources.

2. Retrieval & Context Pipeline (Query Time)

When an query prompt is received, the message is stored in SQLite and vectorized. The query vector is compared against the user's document embeddings in ChromaDB. The top 4 most similar chunks (Cosine Similarity) are retrieved. These segments are structured inside the RAG context framework. The system client extracts conversational histories up to 20 messages, constructs the final prompt, and streams text generations from LLaMA 3.3 via Groq.

3. Security and Multi-User Isolation Model

Passwords are hashed using bcrypt. Access control is maintained using session states. Chat histories are filtered by username in SQLite. Documents are assigned unique UUIDs mapping to user accounts, ensuring ChromaDB retrieves records only belonging to the active user session.

3. API Documentation Reference (Part 1)

Authentication Module (auth.auth)

File Path: `auth/auth.py`

- > `hash_password(plain: str) -> str`
Hashes a plaintext credential with bcrypt using a random salt.
- > `verify_password(plain: str, hashed: str) -> bool`
Checks password string inputs against stored database hashes.
- > `create_user(username: str, password: str, email: str = None) -> dict`
Inserts records to SQLite user table, validating uniqueness.
- > `authenticate_user(username: str, password: str) -> Optional[dict]`
Checks username existence and password hashes, returning metadata.

Document Parser Pipeline (rag.loader)

File Path: `rag/loader.py`

- > `load_file(filepath: str) -> str`
Main dispatcher. Determines type and invokes internal parser (PDF, DOCX, CSV, TXT, MD).
- > `_load_pdf(path: str) -> str`
Extracts text across PDF pages using pdfplumber, joining them with double-newlines.

Text Chunker (rag.chunker)

File Path: `rag/chunker.py`

- > `chunk_text(text: str, chunk_size: int = 500, overlap: int = 50) -> List[str]`
Splits documents into overlapping chunks based on word boundaries.

3. API Documentation Reference (Part 2)

Embedding Engine (rag.embedder)

File Path: rag/embedder.py

- > **get_model()** -> **SentenceTransformer**
Loads the embeddings model as an in-memory singleton. Defaults to 'all-MiniLM-L6-v2'.
- > **embed_texts(texts: List[str])** -> **List[List[float]]**
Vectorizes batches of text strings into 384-dimensional dense arrays.

ChromaDB Connector (rag.vector_store)

File Path: rag/vector_store.py

- > **add_chunks(chunks: List[str], embeddings: List[List[float]], source: str, doc_id: str)**
Upserts batches of text blocks and float arrays to ChromaDB.
- > **query_chunks(query_embedding: List[float], top_k: int = 4)** -> **List[dict]**
Executes similarity searches based on Cosine distances, returning context and scores.

LLM Controller (llm.groq_client)

File Path: llm/groq_client.py

- > **chat(messages: List[dict], context: str = '', stream: bool = True)** -> **Generator[str]**
Configures prompts and pipes inputs to LLaMA 3.3 on Groq. Handles chunk streaming.
- > **trim_history(messages: List[dict])** -> **List[dict]**
Limits dialog payloads to the last 20 records while preserving user-assistant message pairs.

SQL Database Adaptor (utils.helpers)

File Path: utils/helpers.py

- > **init_db()** -> **None**
Initializes relational tables documents and chat_history. Automatically performs username migrations.
- > **get_history(session_id: str, username: str = None)** -> **List[dict]**
Retrieves the conversation logs for a session, filtering by username if provided.

4. Deployment & Operation Manual

Environment Configuration (.env)

Create a '.env' file in the workspace directory with these configurations:

```
GROQ_API_KEY=gsk_your_actual_key_here
CHROMA_DB_PATH=./chroma_store
EMBED_MODEL=all-MiniLM-L6-v2
TOP_K_CHUNKS=4
LLAMA_MODEL=llama-3.3-70b-versatile
```

Execution Instructions

To launch the secure multi-user chatbot, initialize your python virtual environment and run the Streamlit compiler:

```
pip install -r requirements.txt
streamlit run app.py
```

Operational Notes & Recommendations

1. Model Tuning: You can substitute LLaMA 3.3 70B for LLaMA 8B or Mixtral 8x7B by editing LLAMA_MODEL.
2. Storage Migration: The relational database path defaults to SQLite. For production workloads with thousands of concurrent users, switch the db driver to PostgreSQL inside utils/helpers.py.
3. Local Vector Search: ChromaDB indexes reside inside the workspace. Do not delete './chroma_store' unless you want to rebuild the document database from scratch.